

GIT COMMANDS

1. Configuration & Setup

Command	Description
<code>git config --global user.name "Your Name"</code>	Set your name for commits
<code>git config --global user.email "you@example.com"</code>	Set your email for commits
<code>git config --list</code>	View current configuration settings
<code>git config --global alias.co checkout</code>	Create a shortcut (alias) for commands
<code>git init</code>	Initialize a new Git repository
<code>git clone <repo-url></code>	Clone an existing repository

2. Working with Files

Command	Description
<code>git status</code>	Show the current status of the working directory
<code>git add <file></code>	Stage changes for commit
<code>git add .</code>	Stage all changes in the working directory
<code>git reset <file></code>	Unstage a file before commit
<code>git reset --hard</code>	Reset everything to last committed state
<code>git commit -m "message"</code>	Commit staged changes with a message
<code>git commit --amend -m "new message"</code>	Modify the last commit message

3. Branching & Merging

Command	Description
<code>git branch</code>	List all branches

<code>git branch <branch-name></code>	Create a new branch
<code>git checkout <branch-name></code>	Switch to a different branch
<code>git checkout -b <branch-name></code>	Create and switch to a new branch
<code>git merge <branch-name></code>	Merge another branch into the current branch
<code>git branch -d <branch-name></code>	Delete a local branch
<code>git push origin --delete <branch-name></code>	Delete a remote branch

4. Remote Repositories

Command	Description
<code>git remote -v</code>	List remote repositories
<code>git remote add origin <repo-url></code>	Link local repo to a remote repo
<code>git push origin <branch-name></code>	Push changes to the remote repository
<code>git push --force</code>	Force push changes (overwrites remote history)
<code>git pull origin <branch-name></code>	Fetch and merge changes from remote repo
<code>git fetch origin</code>	Fetch remote changes without merging

5. Viewing History

Command	Description
<code>git log</code>	View commit history
<code>git log --oneline --graph --all</code>	View commit history as a graph
<code>git log --author="name"</code>	View commits by a specific author
<code>git log --grep="keyword"</code>	Search commit messages
<code>git diff</code>	View unstaged changes
<code>git diff --staged</code>	View staged changes
<code>git show <commit-hash></code>	View details of a specific commit

6. Undoing Changes

Command	Description
<code>git checkout -- <file></code>	Discard changes in a file
<code>git reset HEAD <file></code>	Unstage a file
<code>git revert <commit-hash></code>	Undo a commit by creating a new commit
<code>git reset --hard HEAD~1</code>	Delete the last commit
<code>git reflog</code>	View recent changes to HEAD and recover lost commits

7. Stashing Changes

Command	Description
<code>git stash</code>	Save uncommitted changes temporarily
<code>git stash list</code>	View saved stashes
<code>git stash apply</code>	Apply the latest stash
<code>git stash pop</code>	Apply and delete the latest stash
<code>git stash drop</code>	Delete a specific stash

Bonus: Useful One-Liners

Command	Description
<code>git shortlog -s -n</code>	Show commit count by author
<code>git grep "function_name"</code>	Search for text in files
<code>git blame <file></code>	Show who modified each line in a file
<code>git diff --name-only HEAD~1</code>	Show files changed in the last commit

License Details:

- If you **want flexibility**, use **MIT**.
- If you **want patent protection**, use **Apache 2.0**.

- If you **want all future versions to remain open-source**, use **GPL**.
- If you **want to restrict branding usage**, use **BSD 3-Clause**.
- If your repo is **not code-related**, use **Creative Commons**.